parse_film.py search_api.py

```
43
44  if __name__ == '__main__':
45    import pprint
46
47    person = search()
48    name = parse_name(person)
49    planet= parse_planet(person)
50    film_list = parse_films(person)
51    titles = format_titles(film_list)
52
53    print(titles)
54
55    description = person_description(name, planet, titles)
56
57    print(description)
```

Find No results

99% (56:1)

Terminal

```
(sw) codio@cabinetchef-igloofocus:~/workspace$ python sw_characters/search_api.py
{'birth_year': '19BBY',
 'created': '2014-12-09T13:50:51.644000Z',
 'edited': '2014-12-20T21:17:56.891000Z',
 'eye_color': 'blue',
 'films': ['https://swapi.dev/api/films/1/',
           'https://swapi.dev/api/films/2/',
           'https://swapi.dev/api/films/3/',
           'https://swapi.dev/api/films/6/'],
 'gender': 'male',
 'hair_color': 'blond',
 'height': '172',
 'homeworld': 'https://swapi.dev/api/planets/1/',
 'mass': '77',
 'name': 'Luke Skywalker',
 'skin_color': 'fair',
 'species': [],
 'starships': ['https://swapi.dev/api/starships/12/',
               'https://swapi.dev/api/starships/22/'],
 'url': 'https://swapi.dev/api/people/1/',
 'vehicles': ['https://swapi.dev/api/vehicles/14/',
              'https://swapi.dev/api/vehicles/30/']}
(sw) codio@cabinetchef-igloofocus:~/workspace$ ls
parse_film.py  sw_characters
```

Guide

Collapse

Parsing the Films

Parsing Film Titles

Activate the virtual environment.

```
conda activate sw
```

The next thing to do is to get a list of the films in which the character appears. Film titles are stored as a list of URLs. These need to be converted into a list of strings. Create the `parse_films` function and pass it the argument `person` which is the general dictionary of the character you got from the API. Set `film_urls` to the value associated with `'films'`, which will be a list of URLs. Using a list comprehension create a list of strings. Use the helper function `fetch_title` to convert the URL to a string.

```
def parse_films(person):
  film_urls = person.get('films')
  films =[fetch_title(film_url) for film_url in film_urls]
  return films
```

The `fetch_title` function calls the API with each film's URL and converts the results into JSON. Then get the value associated with the `'title'` key. Return this string back to the list comprehension above.

```
def fetch_title(url):
  film_json = requests.get(url).json()
  film_title = film_json.get('title')
  return film_title
```

Testing the Function

In the testing section, create the variable `film_list` and call the `parse_films` function. This function should return a list of strings.

```
if __name__ == '__main__':
  import pprint

  person = search()
  name = parse_name(person)
  planet= parse_planet(person)
  film_list = parse_films(person)

  pprint.pprint(film_list)
```

In the terminal, run the script to test your work.

```
python sw_characters/search_api.py
```

You should see the following output:

```
['A New Hope',
 'The Empire Strikes Back',
 'Return of the Jedi',
 'Revenge of the Sith']
```

Formatting the Film Titles

So the film titles are now stored as strings, but they are in a list. We want to have a single string with all of the film titles. To improve readability, we want each title on its own line. We also want to indent two spaces and preface the title with an `*`. Create the `format_titles` function that takes a list of movie titles. With a list comprehension, add a newline character (`\n`) to the end of each element in the list. Use the string method `join()` to create a string of all of the titles in the list, placing `' * '` between each element. Add `' * '` to the beginning of the string so that all elements have the same indentation before the title. Return a single string with all of the film titles, indentation, and newline characters.

```
def format_titles(titles):
  new_lines = [title + '\n' for title in titles]
  formatted_titles = '  * ' + '  * '.join(new_lines)
  return formatted_titles
```

Testing the Function

In the testing section, create the variable `title` and call the function `format_titles`. Printing this variable should display an indented movie title on each line.

```
if __name__ == '__main__':
  import pprint

  person = search()
  name = parse_name(person)
  planet= parse_planet(person)
  film_list = parse_films(person)
  titles = format_titles(film_list)

  print(titles)
```

In the terminal, run the script to test your work.

```
python sw_characters/search_api.py
```

You should see the following output:

```
  * A New Hope
  * The Empire Strikes Back
  * Return of the Jedi
  * Revenge of the Sith
```

Building the Description

So far, the functions we built helped to create the components needed to build the discussion for each character. The `person_description` function builds the final description. Using an f-string, name the character, where they are from, and the movies in which they appear. Return this string.

```
def person_description(name, planet, titles):
  description = f'{name} is from the planet {planet}. They appear in the following
  return description
```

Testing the Function

In the testing section, create the `description` variable and call the `person_description` function. Pass `name`, `planet`, and `titles` to the function. Your script should print the full description of a character from Star Wars.

```
if __name__ == '__main__':
  import pprint

  person = search()
  name = parse_name(person)
  planet= parse_planet(person)
  film_list = parse_films(person)
  titles = format_titles(film_list)
  description = person_description(name, planet, titles)

  print(description)
```

In the terminal, run the script to test your work.

```
python sw_characters/search_api.py
```

You should see the following output:

```
Luke Skywalker is from the planet Tatooine. They appear in the following films:
  * A New Hope
  * The Empire Strikes Back
  * Return of the Jedi
  * Revenge of the Sith
```

▼ **Code**

Your code should look like this:

```
import requests

def search(search_term='luke'):
  base_url = 'https://swapi.dev/api/people/?search='
  search_url = f'{base_url}{search_term}'
  resp = requests.get(search_url)
  resp_json = resp.json()
  if resp_json.get('results'):
    return resp.json()['results'][0]
  else:
    return None

def parse_name(person):
  name = person.get('name')
  return name

def parse_planet(person):
  planet_url = person.get('homeworld')
  resp = requests.get(planet_url)
  planet = resp.json().get('name')
  return planet

def parse_films(person):
  film_urls = person.get('films')
  films =[fetch_title(film_url) for film_url in film_urls]
  return films

def fetch_title(url):
  film_json = requests.get(url).json()
  film_title = film_json.get('title')
  return film_title

def format_titles(titles):
  new_lines = [title + '\n' for title in titles]
  formatted_titles = '  * ' + '  * '.join(new_lines)
  return formatted_titles

def person_description(name, planet, titles):
  description = f'{name} is from the planet {planet}. They appear in the following
  return description

if __name__ == '__main__':
  import pprint

  person = search()
  name = parse_name(person)
  planet= parse_planet(person)
  film_list = parse_films(person)
  titles = format_titles(film_list)

  print(titles)
```

Deactivate the virtual environment.

```
conda deactivate
```

Lab Question

What does the `clean` command do in Conda?

- ○ Formats your code so that it is easier to read.
- ○ Removes any packages installed from PyPI.
- ○ Removes unused files from all virtual environments.
- ◉ Removes unused files from a virtual environment.

The clean command removes unused files from the virtual environment that is currently activated. It does not affect other virtual environments.

Check It!

Next